面向 LLM 的架构设计：什么是真正的 AI Friendly 架构?

原创 营销&交易技术 大淘宝技术 2026年6月1日 01:20 浙江 47人



2025年成为AI智能体(Agentic AI)元年，传统工程架构面临与AI"不确定性"的冲突。AI Friendly架构通过三范式实现演进：1)确定性→概率性，将输出收敛至安全区间；2)结构化→语义化，基于意图而非格式响应；3)静态→动态，从规则转向规划。核心能力包括Multi-Agent系统、Context Engineering(上下文工程)、AI Friendly API及AI可观测体系。实际应用中，AI审核准确率达95.7%，AI答疑系统CogentAI问题解决准确率超98%，为业务带来80%以上效率提升。架构升级需根据业务深度需求，避免"为用AI而用AI"。

01 引言

25年伊始，智源研究院提出了《十大AI技术趋势》，2025年成为AI智能体（Agentic AI）的元年，更通用、更自主、深入工作与生活场景的智能体成为各行各业争相追逐的目标。

站在新年，回望AI应用的发展历程，随着一些大模型明星产品的出现，可以清晰的感知到行业对于AI应用形态的理解也已经越发深入：

- 从Chatbot、Copilot到AI Agent，AI从提供建议，跨度到可以辅助完成任务；
- 随着AI Agent应用编排框架不断收敛、Agentic AI概念的提出，从更强调产品概念的AIAgent，跨度到可以独立完成任务，集自主规划、决策、执行于一身，更强调应用智能程度的Agentic AI。

可以预见，此后也将看到智能化程度更高、对业务流程理解更深的多智能体系统在应用侧的落地，而对处于时代弄潮之巅的AI工程师来说，如何将工程架构完成从**传统工程架构向AI Firendly架构**的转变，已然成为进入AI时代的重要门票。

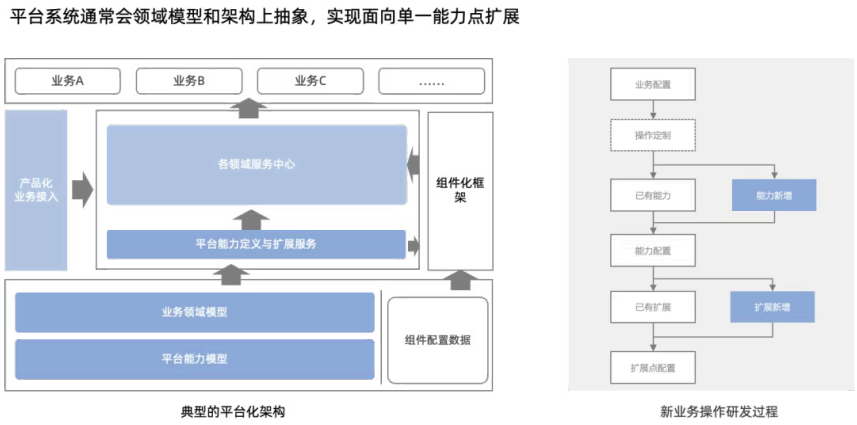
PS：由于笔者所属工作职责为业务开发，所以接下来会以我所在营销业务中探索相对深入的AI应用场景(答疑、审核、建联)为例，循序渐进的阐述AI Friendly架构的设计理念，并结合所使用的**ideaLab平台**的基建能力，讲解AI Friendly架构中核心能力的实现过程。全篇接近万字手搓内容，也参考了很多优秀的文章，希望对正在面临AI应用转型而无从下手的读者们有所帮助。

传统的工程架构按照服务主体的不同，通常可以划分为**业务型架构**和**平台型架构**，这一点在曾经贯彻“**大中台小前台**”战略的阿里来说，表现的尤为明显。

传统平台型架构

对于需要提供平台服务的“**大中台**”来说，由于需要承接集团内不同的业务场景，其系统设计核心是领域模型的标准化与业务流程的抽象化，通过统一抽象实现新增功能可配置、可组合、可沉淀，使相似场景的支撑效率呈“摩尔定律”式提升，从而达到80%~90%功能的复用。

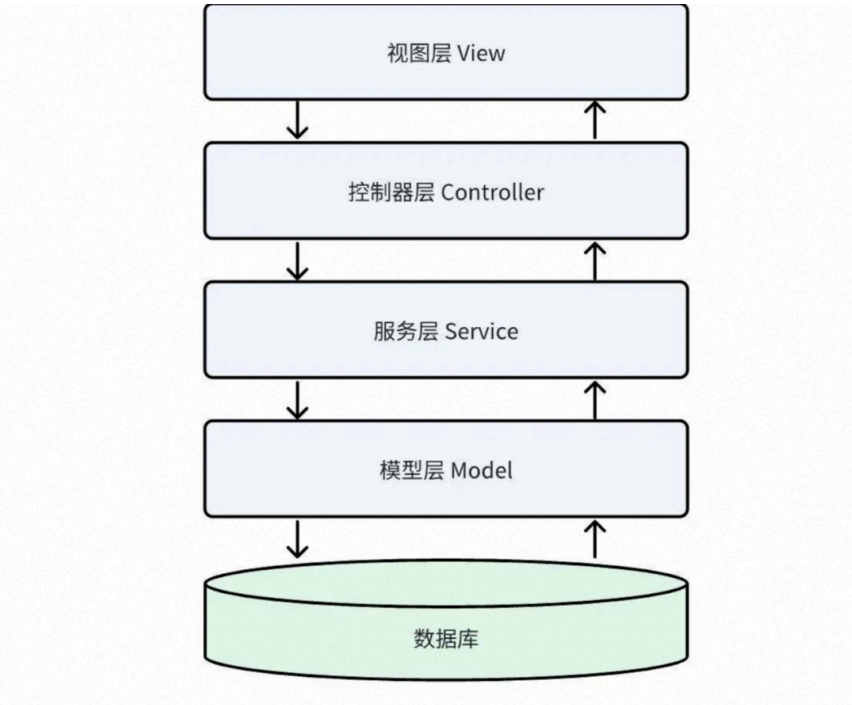
集团内典型的中台如：商品发布系统、营销报名系统等，均是采用这种平台型架构设计思想来实现的，架构示例图如下：



DDD架构的抽象与演进

传统业务型架构

对于需要快速发展的“**小前台**”来说，快速响应、快速上线、快速试错的敏捷开发方式，注定了其系统的架构设计不会如平台型这般复杂，简单成熟的MVC便成为大多数业务型系统所使用的架构——在基础的分层思想上，通过引入恰当的设计模式和数据结构来支持业务的快速发展。典型的业务型架构示例图如下：



MVC架构的抽象与演进

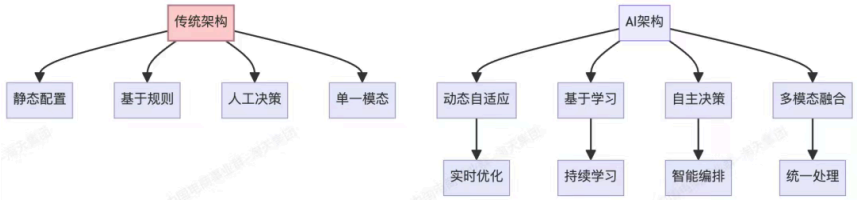
AI时代的冲突

但无论是基于DDD思想的平台型架构还是基于MVC思想的业务型架构，在设计之初，我们其实都是秉承着“以人为本”的理念去进行设计的，这就决定了我们的系统更趋向于处理**确定性**的逻辑，即要求——输入格式规范、输出内容可预测、基于预定义的流程来执行、依赖于规则和配置。

但AI天生具有概率性和涌现性，引入AI的系统也应该具备——“动态自适应”、“自主决策与动态编排”、“持续学习与实时优化”这种处理**不确定性**的能力。所以当以“不确定”为基础的AI 遇见 以“确定性”为基础的传统架构时，自然而然便产生了难以调和的矛盾，例如：

- 当AI的输出没有遵循既定的schema格式时，传统架构便束手无策，只能默默报错；
- 当流程/工具/函数需要根据上下文动态选择时，传统架构便无计可施，只能望“规则”兴叹；
- 当以高延迟、低吞吐为特点的Agent进入到以低延迟、高吞吐为设计原则的传统架构中，各种超时、可用性的问题便层出不穷；

可以看得出来，如何实现传统的工程架构向AI Firendly架构的演进，适配AI时代的“数据结构与算法”，已经成为AI工程师们迫在眉睫的事情。



传统架构与AI架构运行范式的区别

03 演进：从“传统”到“AI Friendly”

演进的三范式

实现传统架构向AI Friendly架构的升级，其实核心逻辑并不复杂，归结为一点就是：**赋能传统工程以驾驭“不确定性”的能力。**

需要着重强调的是，AI Friendly架构并非对传统工程经验的全盘否定，而是在我们过去十几年构建的坚实工程地基之上，为应对“不确定性”所进行的一次精准架构升级，这一演进过程主要包含三个关键维度的范式转变：

1、从确定性到概率性

- 传统工程建立在确定性逻辑之上。系统输出严格遵循 y=f(x) 的映射关系，结果是非黑即白的二元存在：代码要么运行成功，要么抛出异常；
- AI工程则运行于概率空间之中。系统输出是大模型、提示词、上下文与环境共同“涌现”的结果。架构设计的核心目标不再是追求零误差，而是通过 RAG 增强、提示词或上下文工程及评测机制，**将这种概率性的输出收敛至业务与用户可接受的“安全区间”之内。**

2、从结构化到语义化：

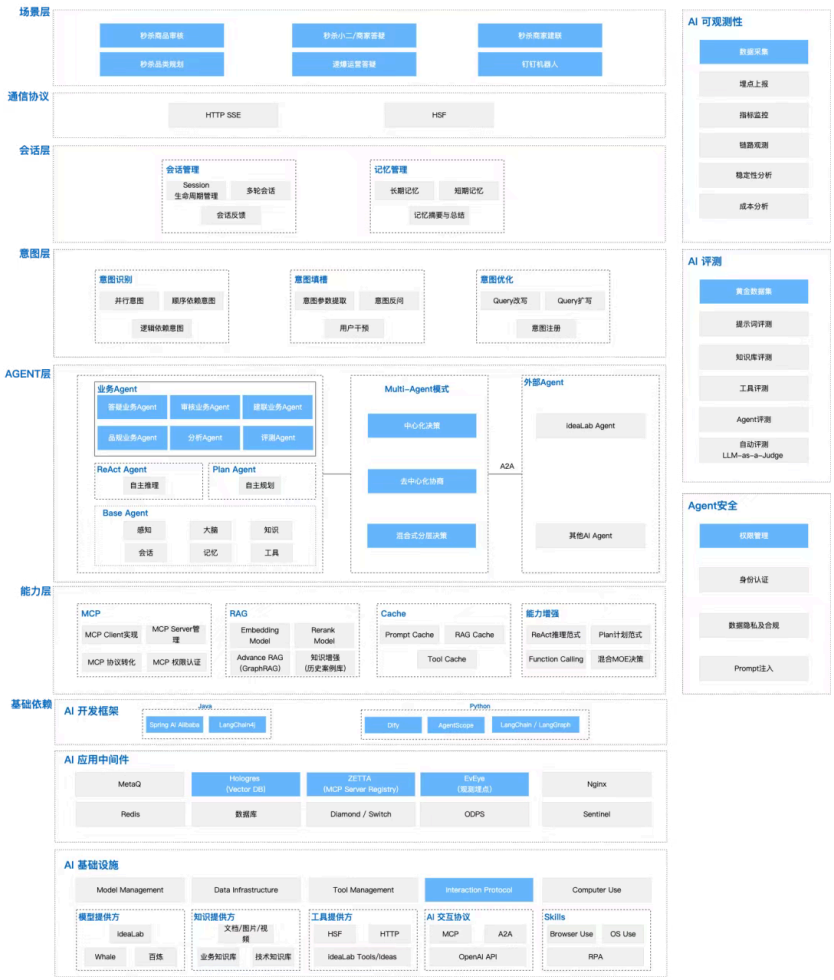
- 传统工程遵循严格的结构化约定，系统输入数据必须精确符合预定义的 Schema（如 JSON 字段类型），任何越界的输入都会触发校验失败，系统边界是一堵刚性的墙；
- AI工程则拥抱语义化的柔性交互，系统能够直接理解自然语言、非结构化数据等模糊输入的意图，**架构设计的核心目标是基于“意图”而非“格式”进行响应**，系统边界演变为一层弹性的膜。

3、从静态到动态

- 传统工程基于预定义流程（Workflow）开发，系统执行路径依赖于硬编码的逻辑判断（if-else）或规则，系统行为是可穷举、可验证的。虽然规则也可以灵活配置，但本质上是静态的、被动的；
- AI工程能够基于模型作出决策，系统具备根据当前环境和目标进行推理的能力，可以无需人工干预地拆解任务、调用工具并响应未知的变化，属于动态的、智能化的系统。**架构设计的核心目标需要从“规则”转向“规划”，即通过赋予系统更高的自主性，从而实现任务的自主决策与智能编排。**

AI Friendly架构大图

基于上述演进的三范式，笔者所构造的AI Friendly架构大图设计如下，这里先对架构大图做一个简单的介绍，结合业务系统的实现在后文中也会有所展开：



基础依赖层即架构所需的底层能力和数据支持，与传统架构不同的是，AI Friendly架构会更关注“模型”、“知识”、“工具/Skills”的底层能力和数据支持。在笔者的AI业务系统中：

- 模型管理（Model Management）依托于**ideaLab**提供的API，可以基于OpenAI协议调用各大厂商提供的不同版本的大模型，无需关注不同模型接入的适配逻辑；
- 知识管理（Data Infrastructure）依托于**ideaLab**提供的知识库能力，可以将不同来源的知识（如钉钉、语雀）进行向量化存储和检索，无需关注Embedding Model、向量数据库的接入逻辑；
- 工具管理（Tool Management）依托于**ideaLab**、**ZETTA**共同提供的能力，其中：
  - ZETTA是HSF团队开发的MCP管理平台，可以基于MCP协议将HSF接口快速转化为MCP Server；
  - ideaLab则是基于MCP协议实现了MCP Client，可以通过配置MCP Server URL在ideas中实现工具调用（Function Calling），当然也可以直接配置HSF/HTTP接口来实现工具调用；

- 同时我们基于RPA实现了Computer Use的Skills，包括浏览器的使用、操作系统(如钉钉)的使用，虽然在架构图中单独画了一个模块，但本质上也是属于工具管理的范畴；
- 另外，由于笔者所在团队都是以Java开发为主，所以采用了Spring AI Alibaba作为开发框架来做实现模型、知识、工具等基础依赖的管理，以及上层Agent、意图、会话等能力的建设，同时结合ideaLab等平台能力简化了**能力层**中MCP和RAG能力的建设。读者也可基于自己的开发环境，选择适合自己的开发框架以及MCP、RAG等能力层的实现方式。

Agent层、意图层、会话层是AI Friendly架构独有的三层模型，也是实现传统架构从确定性到概率性、从结构化到语义化、从静态到动态转变的关键所在。其中：

- **Agent层**负责实现不同业务场景下Agent的构建和管理。在笔者看来，未来的Agent可以分为两类：一类是具备动态规划解决问题能力的Agent，另一类是具有固定流程编排的Agent（其实更应该叫做AI Workflow）。基于此，在笔者的AI业务系统中，构建了三种Agent的实现方式，分别为：BaseAgent、ReActAgent和PlanAgent，其中：
  - BaseAgent默认为普通ChatBot或AI Workflow的实现，不具备动态规划能力；
  - ReActAgent和PlanAgent则是基于ReAct推理范式和Plan计划范式实现了自主推理动态规划能力，是真正意义上的Agent；
  - 同时基于一些Multi-Agent的最佳实践模式（如中心化决策和去中心化协商），实现了不同Agent（包括外部Agent）之间的合作；
- **意图层**负责对任务的真实目的进行识别和处理，实现传统架构从结构化到语义化的转变。但不是所有的业务场景都需要意图层，一些简单的任务也可以不进行意图识别而直接使用相关Agent，只有当任务需要多意图识别时（如答疑场景）才需要引入意图层。意图层需要关注“并行意图”、“顺序依赖意图”、“逻辑依赖意图”的处理方式，同时也需要结合Query的改写和扩写来实现意图的优化。
- **会话层**则重点需要关注多轮会话及长短期记忆能力，对于AI工程师来讲，记忆的本质就是上下文工程，其重要程度甚至高于模型本身。一个优秀的模型如果没有适配的记忆，其表现程度甚至不如过时的模型

AI可观测、AI评测、Agent安全这三个模块其实本质上属于质量和稳定性管理，在传统架构中也是属于必不可少的模块，只不过基于AI应用的特点，其可用性SLA的衡量标准也会有所不同，在下文中会做详细讲解，这里就不做过多赘述了。

所有的AI工程都需要进行架构演进吗？

需要强调的是，集团内也有很多基于Workflow搭建AI智能体的平台，如ideaLab，都极大的降低了我们接入AI应用的门槛，并且现在AI应用的绝大部分场景，也基本都是通过AI

WorkFlow的方式来实现。**那么，对于仅需要接入AI WorkFlow获取结果的系统，是否需要向AI Friendly架构演进呢?**

笔者认为，不需要也没必要。对于此类系统的应用场景而言，是将构建好的Agent当作接口来使用，所以只需要关注API调用和结果处理即可，无需关注“记忆管理”、“工具管理”、“上下文工程”、“Multi-Agent调度”、“自主规划”、“AI 可观测性”、“数据采样与评测”等更深层次的AI能力维度，传统的架构设计或平台提供的能力已然满足使用的诉求。只有当所面临的业务场景需要涉及到更深层次的AI应用时，才需要对系统架构进行升级。正如我们提倡不要“为了用AI而用AI”，自然也不需要“为了用AI升级AI架构”，使用适合的架构做合适的系统，才是最重要的。

04 发展：落地AI驱动的业务系统

以上也只是对AI Friendly架构中各层的能力和作用做一个简单的描述，仅靠架构大图的话，想必各位读者还处在知其然但不能知其所以然的地步，所以接下来还会结合AI业务系统落地的实战，来讲解架构中涉及到的核心能力是如何实现的。

业务场景

笔者所属团队为用户场景营销团队，我大概是从去年4月份左右接到探索AI的命题，彼时GPT、Claude、Gemini轮番推出震惊世界的大语言模型，比如红极一时的GPT4o，还有Gemini1.5系列。大家对AI（准确来说是大语言模型）充满了空前的信心，但对可落地场景却一头雾水，不知道AI的边界在哪里，能用AI做什么事情，以及应该怎么使用AI，于是我们也是摸着石头过河，在集团ideaLab等基建平台的肩膀上，逐渐探索和实现了秒杀业务审核、答疑等AI场景的落地，构建了一整套完整的“AI驱动”的业务系统：

- AI审核

在秒杀业务“高频迭代、节奏紧凑、供给海量”的环境下，商品管理已远超传统“审核通过即可上线”的简单逻辑。运营小二需在极短时间内完成优质商品的快速引入、风险商品的精准拦截、在团商品的动态调优以及劣质商品的及时清退。而现有流程高度依赖人工操作：商家报名门槛高、素材准备繁琐；审核环节信息过载、易漏风险；商品上线后缺乏实时监控，问题往往滞后暴露，这一系列断点导致运营人力被大量事务性工作占用，严重制约秒杀货盘的质量与爆发效率。

秒杀AI审核通过构建覆盖商品全生命周期，系统性解决两大核心问题：

1. **审核负担重、风险识别滞后**

   → 基于多模态AI模型实现风险自动分级（自动通过/驳回），提升审核效率，漏审率显著下降。

2. **在团商品“只管上线、不管表现”**

   → 实时巡检类目错挂、sku低价引流、素材风险、蹭大牌等健康度指标，对劣质商品自动预警或下架，守住用户体验底线。

经过一段时间的实践，目前已经做到了AI全自动审核，并通过微调、MOE等能力的建设和优化：

- **AI审核准确率达到95.7%，召回率达到99.1%；**
- 在每天需要审核2-3w报名商品的秒杀业务场景中，为小二带来了**80%以上审核效率提升；**
- 同时对审核场景做了融合与泛化，可以识别出来一些未定义的潜在问题。

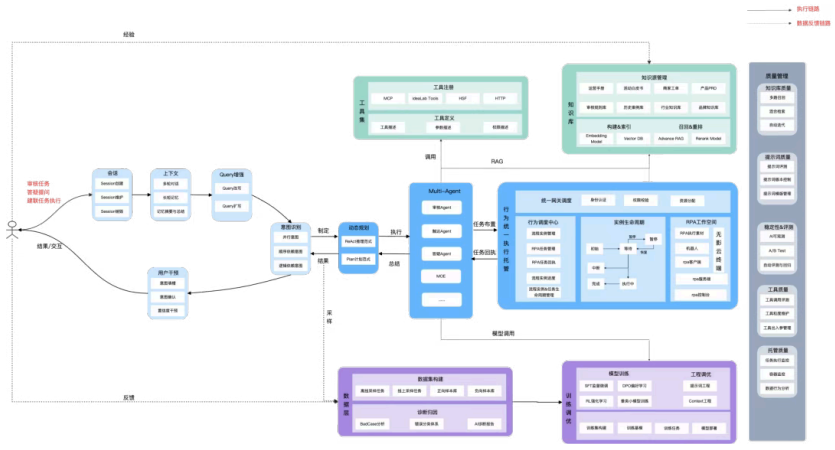
- AI答疑

CogentAI是我们基于秒杀业务答疑场景，做的一款**具备自主规划、推理解决问题能力**的AI助理。与传统QA问答机器人或RAG问答机器人不同，CogentAI可以根据问题进行意图识别、自主规划问题的解决路径、灵活选择工具和知识库使用、并根据结果动态调整计划。

CogentAI可以解决运营/商家提出的某些具体商品/活动的复杂问题，而不仅仅是预设QA或RAG文档的回答，AI问题解决的准确率在98%以上，为使用者带来80%以上的效率提效。

系统能力图谱

基于AI Friendly架构的设计理念，我们构建了AI Friendly的业务系统来承接上述秒杀AI业务场景，系统能力图谱和链路如下图所示：



相信做过AI应用的同学对其中涉及到的能力或多或少都有些了解，一些较为简单的能力笔者就不再赘述了，接下来主要会对一些核心的能力做讲解：

- 从ReAct 到 Multi-Agent

从定义上来看，ReAct 和 Multi-Agent其实是两个完全独立的Agent技术

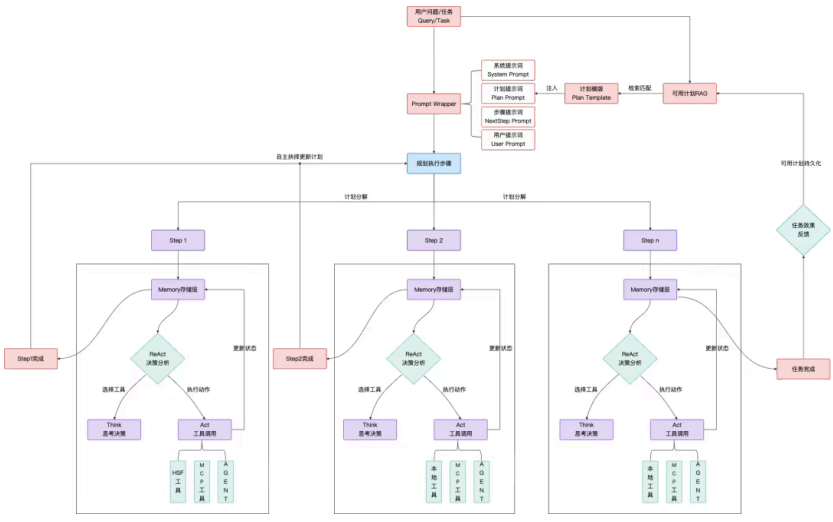
- ReAct是Single-Agent的重要构建方式之一，可以赋予Agent自主推理解决问题的能力；
- Multi-Agent是多Agent之间的交互方式，Agent之间通过对话、消息、共享等方式来协作完成任务；

但二者在实现层面会存在交集，从ReAct到Multi-Agent也可以看作是从Single-Agent到Multi-Agent。

ReAct 范式的实现

ReAct (Reasoning + Acting) 的核心逻辑是让大模型在执行任务时，按照 `思考(Thought)` -> `行动(Action)` -> `观察(Observation)` 的循环来工作。它核心解决的是单个模型无法一次性完成复杂任务的问题，通过ReAct的方式让 Agent 能够“自言自语”地规划，并一步步调用工具、补充上下文来解决问题。

不过从实践来看，虽然ReAct范式可以赋予Agent自主推理解决问题的能力，但由于其每次都是基于当前信息推理下一步的最佳行动，这种单步思考的方式决定了它更擅长解决理性类问题（例如排查一个商品为什么不能报名），对于主观类问题并不能做到很好的解决，所以ReAct范式通常会结合Plan范式来形成Agent的**计划-推理**能力，此时一个复杂任务的完整执行流程如下：



ReAct And Plan范式的结合

- 由Plan产出全局计划，通过沉淀优秀的**计划模版**来解决可能出现的低质量计划问题；

- 由ReAct来执行细分领域的推理，其优秀的推理能力可以在垂直领域表现出更好的效果；

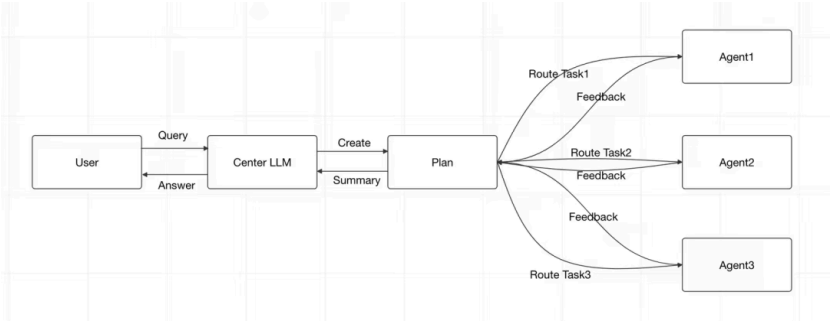
基于此，我们在AI业务系统中也抽象出了ReActAgent和PlanAgent的实现来适配不同的业务场景，这部分在上面的架构大图中也有所体现。

Multi-Agent的实现

Multi-Agent的实现，其实本质就是Agent之间**通信协议**与**协调机制**的实现，即在多个Single-Agent的基础上，建设的分布式自主决策系统。通常按照决策的集中化程度，可以分为三种模式：

架构模式	中心化决策	去中心化协商	混合式分层
通信机制	中心Agent广播	点对点消息	层级路由
决策延迟	低（单决策点）	高（需共识）	中（局部自治）
适用场景	流程固定、时效敏感	动态环境、需弹性扩展	大型/混合任务

在秒杀AI答疑业务场景中，我们就是采用**中心化决策**的模式实现了Multi-Agent。我们将答疑分为多个业务域，例如：商品域、订单域、库存域、报名域、补贴域、素材域等，每个域基于ReAct And Plan范式实现了具备“计划-推理能力”的Agent，由中心Agent统一做意图识别与任务分发，并接受各域Agent的结果反馈推进任务执行，形成“MOE(混合专家)”形态的Multi-Agent：



- 从Prompt Engineering到Context Engineering

相信做过AI应用的同学，都经历过提示词工程的“折磨”：极尽各种提示词的使用模版和技巧，来博取LLM大人的“欢心”。

在当前阶段，提示词工程依然很重要，这是毋庸置疑的。不过笔者在这里想表达的一个观点是，Prompt Engineering始终是一门经验学科，当前提示词的各种使用经验本质上是在弥补大模型能力的不足，当大模型能力越来越强时，一个简单的指令和描述就可以完成任务，过多的提示词反而会成为累赘，所以处于AI时代的工程师们，更应该关注的是Context Engineering：即上下文工程。

ContextEngineering

上下文工程也不是什么高大上的东西，究其根本也离不开“Prompt”和“RAG”。但上下文工程并不是简单的根据用户Query做下RAG检索生成就完事了，而是**如何精心挑选、组织和压缩信息，以便在有限的窗口内，让大模型获得回答问题所需的最优知识**。

通常每个AI应用的上下文工程需要结合具体的业务场景来做定制化的链路，每个AI应用所需要的上下文工程能力也不一而足，例如**在秒杀AI审核业务场景中**，我们建设了两个能力来完善和优化AI审核的上下文：

- 历史审核案例库：通过沉淀历史审核的优秀案例并存入向量数据库，对当前审核商品做图片、标题等维度的相似性向量检索，召回最佳案例提供给大模型做参考，优秀的案例可以为AI审核带来8%左右的准确率提升；
- 混合审核决策：建设多模型投票和置信度机制，通过引入水位有差异的多模型进行多次判断，将投票后的结果给到大模型做参考，结论一致且置信度较高的情况下为AI审核带来10%以上的准确率提升；

当然也会有一些通用的上下文工程能力，例如“长短记忆”、“摘要总结”，这些也都有相对通用的实现，读者可以根据自己的业务场景来构造适配的上下文。

AdvanceRAG

除了一些常规的RAG，一些Advance RAG的技术也可以给Context Engineering带来更好的效果，比如：

- 知识图谱与结构化，可以建立起知识之间的联系，在答疑场景尤为显著，代表技术产品：GraphRAG；
- 动态上下文剪枝（Context Pruning）
- ……

感兴趣的同学可以通过关键词搜索了解下。

- 从REST-ful 到 LLM-ful

知识相关的能力建设讲解完毕后，这里也讲下工具相关的。对于工具而言，核心的能力点就两个：工具的定义和使用：

工具定义

传统工程的开发通常都是面向接口编程，工具接口的名称、入参、出参的定义都是给人看的，所以隐含了很多业务或约定成俗的语义，这些对于大模型是不能理解的；同时为了方便我们通常也会将入参或出参的嵌套做的较深，一些重点的字段信息也淹没在偌大的数据结构中，这些对于大模型也是很难理解的。

而在AI时代，工具的使用主体会从“人”转变成“AI”，所以接口的定义也应该从REST-ful到 LLM-ful，即做**AI Friendly API** 和 **AI Friendly Error**。在秒杀AI答疑业务场景中，

我们对原生的商品、订单、报名等接口做了AI Friendly的改造，主要包括：

- 工具原子化改造：对接口做原子化能力的重新拆分，拆分成适配大模型ReAct推理过程的原子工具；
- 工具出入参改造：对接口的名称和出入参做拟人化调整，接口名称清晰、直接的体现用途，出入参仅保留核心参数和字段，尽量使用平铺的KV对来描述；
- 工具Error改造：对接口可能出错的情况做划分，预期内的情况尽量提供简短的错误描述，方便大模型做推理决策；预期外的情况提供堆栈信息，方便大模型识别错误原因。

工具使用

在Function Calling/MCP/Skill协议相对完善的今天，工具的使用反而成为了最简单的一步，目前各大平台均有成熟的能力可以使用，基于平台或框架能力就可以直接补齐工具使用的能力拼图

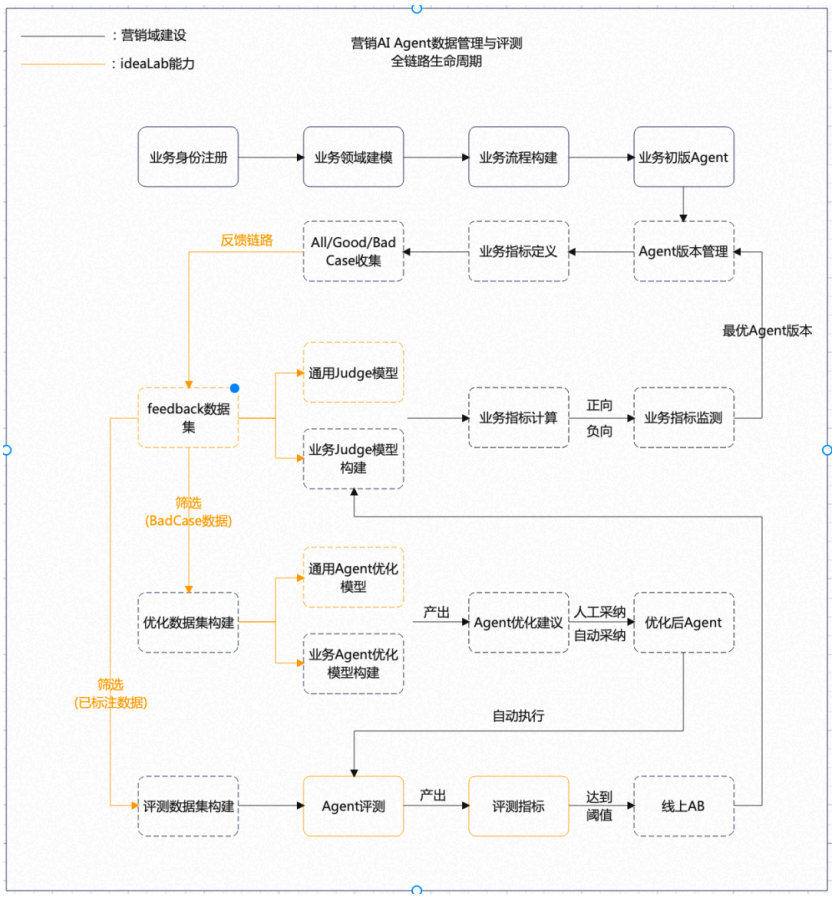
- 从评测 到 AI可观测

AI应用构建完成后，最重要的便是评测和可观测能力的建设：

- 评测即对AI应用的效果保障，通过评测来形成Agent更新迭代的飞轮；
- 可观测即对AI应用的质量保障，通过埋点&监控来对Agent的稳定性进行分析

数据集管理与评测

从笔者的实践来看，评测所花费的时间甚至要超过Agent构建所花时间的两倍以上，所以形成一套基于黄金数据集的可持续评测链路，是每个业务都需要重点建设的能力。在笔者的AI业务系统中，数据集管理与评测链路如下：



整体链路通过“线上数据采样 -> 样本集构建 -> 评测（自动 、人工） -> 优化（工程优化 、模型微调 ） -> 线上AB -> 指标观测”的正向循环链路，形成AI应用迭代更新的飞轮。

AI可观测

在集团内，我们有 EagleEye 、Sunfire这样的可观测产品帮助我们对传统应用做分析和监控，其可观测性关注的是**微服务之间的请求延迟、错误率和资源使用等指标**。

但在AI时代，面对大模型的语义性和不确定性，可观测更应该关注的是**大模型及Agent相关的指标**，如：Agent执行路径、首Token响应时间（TTFT）、Token消耗与成本、TPM、QPM等，因此AI可观测性还必须深入到决策链、上下文等LLM或Agent层面。

从效果保障的角度来看，评测和可观测也是密不可分的，上面阐述的“数据集管理与评测链路”更多的还是从**执行结果**维度来衡量效果，但对于真正的Agent，还应该从**执行路径**维度来进行评测：例如ReAct Agent的推理路径 和 Plan Agent的执行计划过程是否优秀，这些也离不开AI可观测的能力。

而且从长远建设的角度来看，AI可观测未来也会和测试紧密结合，完成AI应用上线前进行“**回归测试**”，实现AI应用的上线标准制定及发布流程接入：



05 结语

原以为一篇文章便可以详细的介绍完笔者于AI实践中的所得，但洋洋洒洒了近万字后也是发现，AI本身便已具备了无限的可能，更不用说叠加“架构”、“业务”等任一复杂领域后又具备了万般变化，自己所做之探索也不过是AI浪潮中的沧海一粟，所讲之内容也不过是浅尝辄止，难以达到事无巨细的地步，也许笔者今日探索得来的经验，过几日就成了过时或众所周知的能力。

AI的发展仍处于日新月异的地步，没有所谓的专家和资深者，人人都还是求学者。所以在这里也欢迎各位读者就文章中的不同意见或实践中所遇到的问题进行讨论，一起探索AI应用更深次的形态，一起迎接Agentic AI的元年。

06 团队介绍

本文作者久游，来自淘天集团–营销&交易技术团队。本团队承担淘天电商全链路交易技术攻坚，致力于通过技术创新推动业务增长与用户体验升级。过去一年主导了多个高价值项目，包括：支撑618、双11、春晚等亿级流量洪峰、构建业界领先的全网价格力体系、承接淘宝全面接入微信支付、搭建集团最大的AI创新平台–ideaLAB，支撑淘宝秒杀等创新业务的高速增长。

拓展阅读



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 8035

留言

写留言

还没有留言，来写首评